



# University of Asia Pacific

Department of Computer Science and Engineering

## CSE 316: Microprocessors and Microcontrollers Lab

### LAB REPORT

---

**Experiment Number:** 07

**Experiment Title:** Remote AC Bulb Control

**Submitted by:**

**Name :** Sunandan Das

**Student ID :** 23101218

**Section :** D2

**Submitted to:**

**Zaima Sartaj Taheri**

Lecturer,

Department of Computer Science and Engineering

**Date of Submission:** 24/04/2026

## **1. Experiment Name :** Remote AC Bulb Control

### **2. Objective:**

The objectives of this project are to design and implement a remote-controlled AC bulb switching system using an Arduino Uno and an infrared (IR) receiver. The project aims to decode IR remote signals and use them to control the switching of an electrical load, specifically an AC bulb. Through this experiment, the principles of IR communication are explored to understand how wireless control can be applied to home automation systems. Furthermore, the control logic for turning the bulb ON and OFF using different remote buttons is simulated and verified to ensure proper functionality. Finally, the experiment provides hands-on experience in interfacing sensors and output devices with a microcontroller, enhancing practical knowledge of embedded systems and wireless control applications.

### **3. Hardware & Software Requirements:**

#### **Hardware:**

- 1 × Arduino Uno R3
- IR Receiver Sensor
- 1 × IR Remote Control
- 1 × Relay Module (SPDT)
- 1 × AC Light Bulb
- 1 × 9V Battery
- 1 × Breadboard
- Jumper Wires

#### **Software:**

- Arduino IDE
- Tinkercad

#### 4. Circuit Diagram:

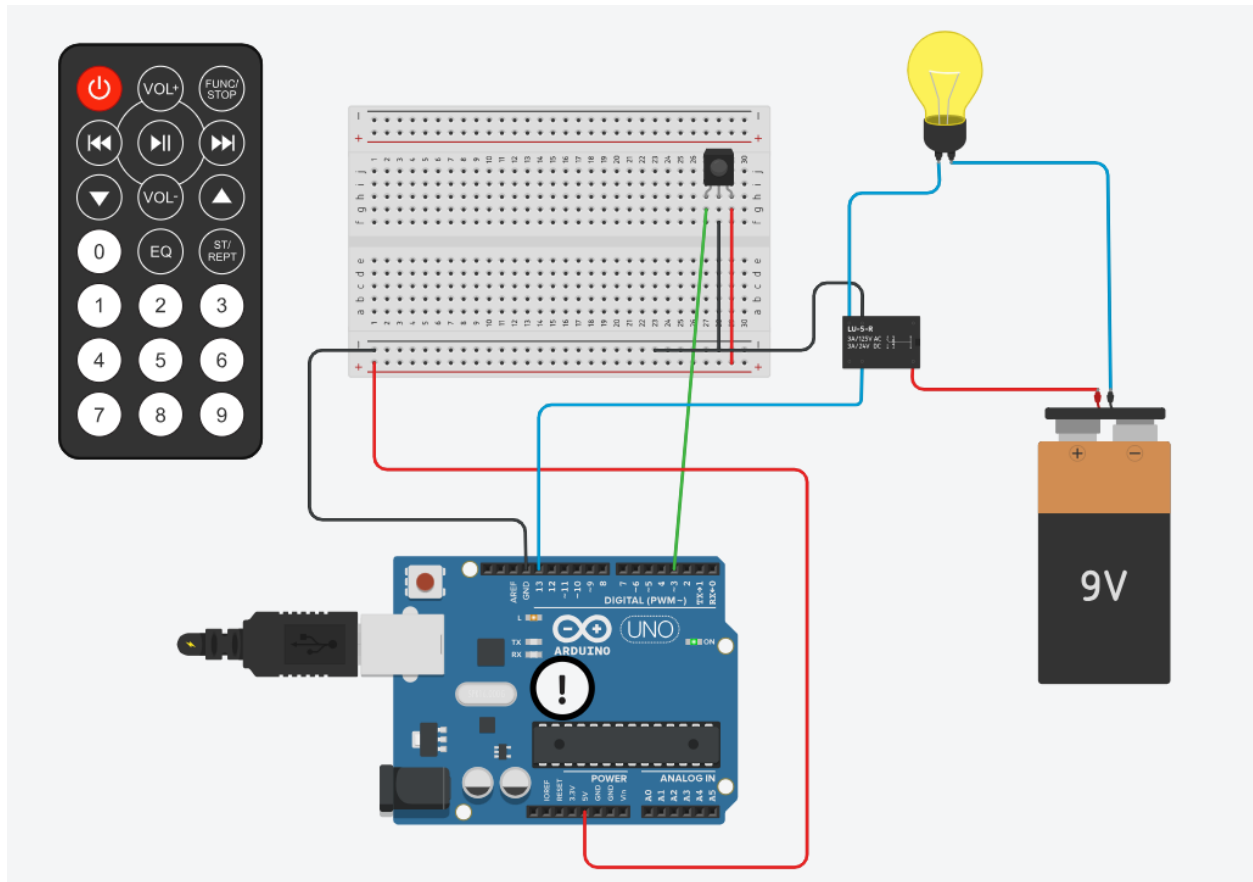


Figure 1: Circuit diagram of a Remote AC Bulb Control

## 5. Assembly Program:

```
// Remote AC Bulb Control
#include <IRremote.hpp>

const int rcvPin = 3; // IR receiver pin
const int ledPin = 13; // LEDs controlled by buttons
void setup(){
  Serial.begin(9600);

  IrReceiver.begin(rcvPin);

  pinMode(ledPin, OUTPUT);
  digitalWrite(ledPin, LOW); // Making sure LEDs start OFF

  Serial.println("IR receiver ready. Press buttons on your remote...");
}
void loop(){
  if (IrReceiver.decode()){
    unsigned long value = IrReceiver.decodedIRData.decodedRawData;

    Serial.print("Button code: 0x");
    Serial.println(value, HEX); // Print code in HEX for reference
    // Match button codes with LEDs
    switch (value) {
      case 0xEF10BF00: // "Button 1" code
        Serial.println("Button 1 pressed");
        digitalWrite(ledPin, HIGH);
        break;
      case 0xFF00BF00: // POWER OFF
        Serial.println("Power Button pressed");
        digitalWrite(ledPin, LOW);
        break;
      default:
        Serial.print("Unknown button: 0x");
        Serial.println(value, HEX);
        break;
    }
    IrReceiver.resume(); // Ready for next signal
    delay(50); // Small delay to stabilize reading
  }
}
```

## 6. Observations:

Upon execution in Tinkercad, the following observations were noted:

### a) Button 1 pressed (Bulb ON):

- Bulb turns on when button 1 is pressed on the IR Remote
- Serial Monitor outputs corresponding text

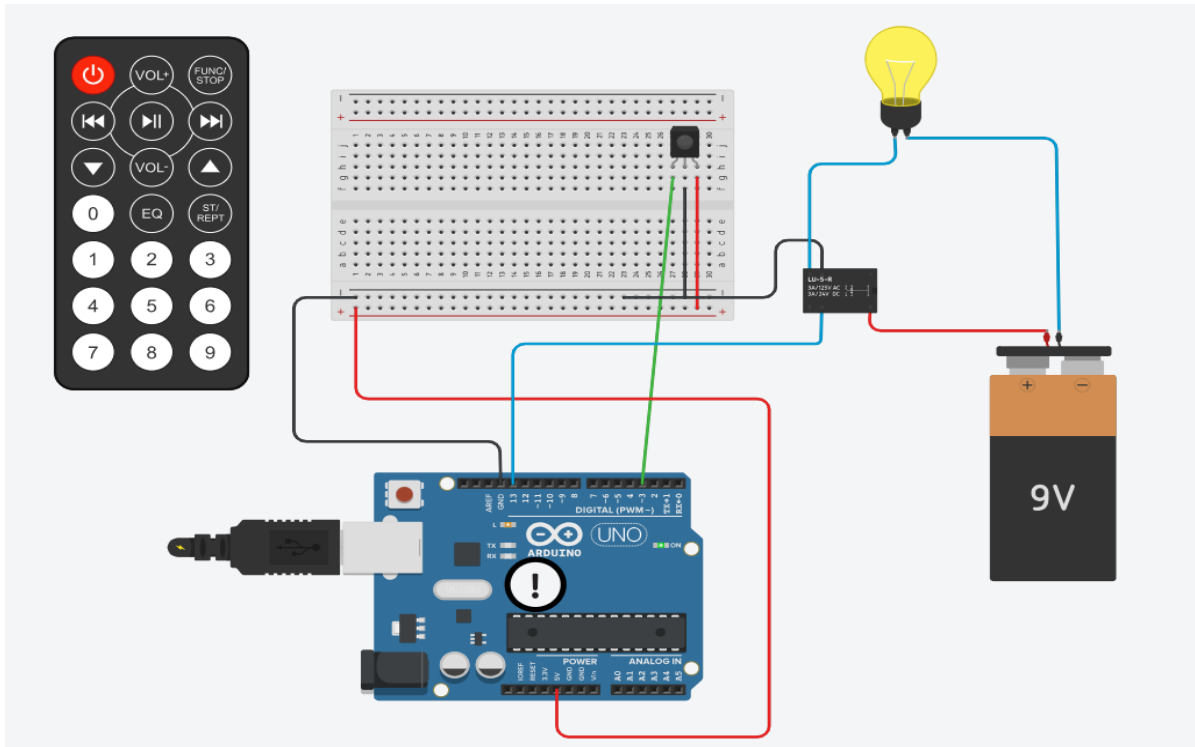


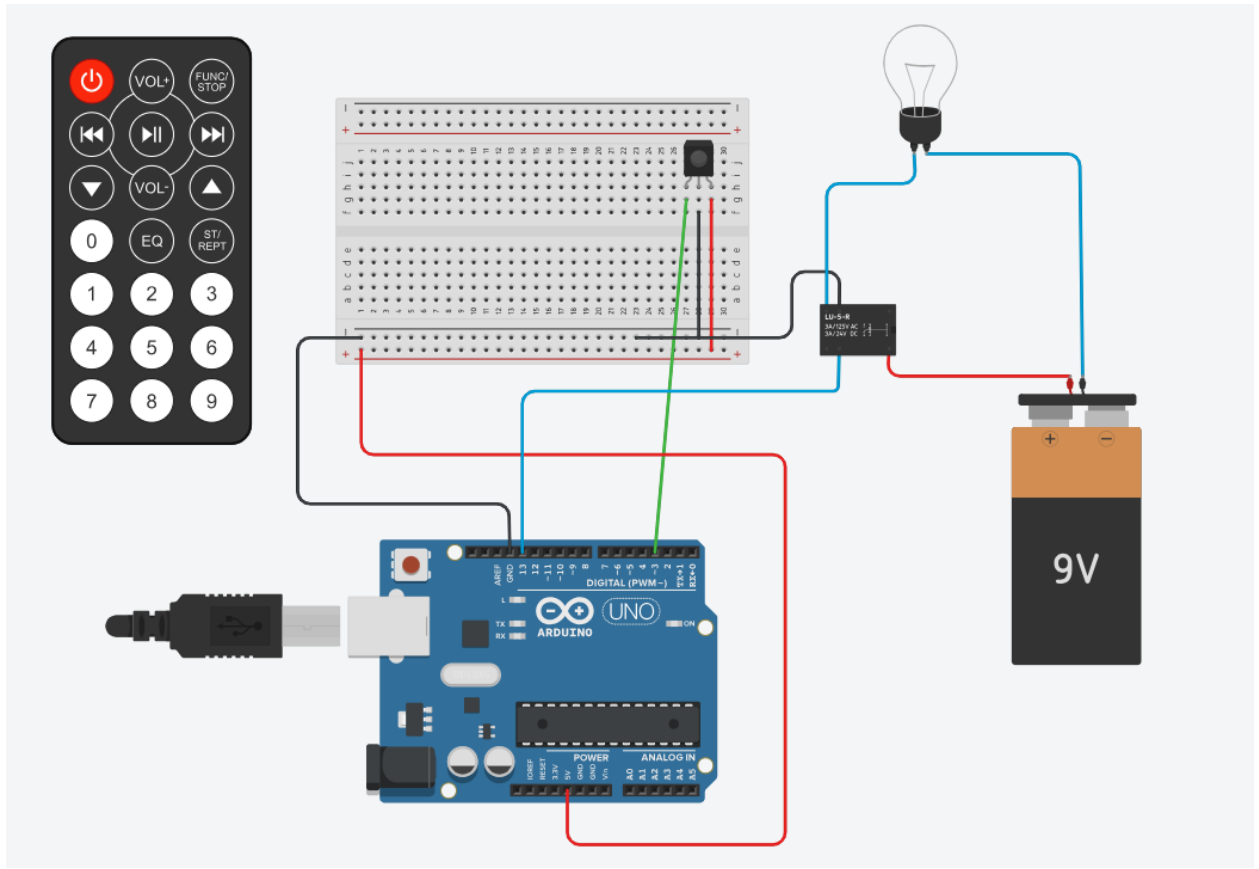
Figure 2: Bulb ON on pressing button 1

```
Button code: 0xEF10BF00  
Button 1 pressed
```

Figure 3: Serial Monitor Output for Case A

### b) Power Button Pressed (Bulb OFF):

- Bulb turns off when power button is pressed on the IR Remote
- Serial Monitor outputs corresponding text



**Figure 4: Bulb OFF on pressing power button**

```
Button code: 0xEF10BF00
Button 1 pressed
Button code: 0xFF00BF00
Power Button pressed
```

**Figure 5: Serial Monitor Output for Case B**

The system's core function is to effectively decode signals from the remote. The components used played a great role in the execution of this project:

1. The IR receiver continuously monitors for incoming signals.
2. Each press of a remote button generates a unique hexadecimal code which the system correctly decodes and displays on the Serial Monitor in real-time. This ensures accurate identification of the intended command.
3. The system is programmed to respond to specific remote commands to control the light bulb:
  - Pressing Button 1 triggers the Arduino to switch the bulb ON

- Pressing the Power button triggers the Arduino to switch the bulb OFF.
  - Any other button press is acknowledged on the Serial Monitor with the message "Unknown button" but does not change the bulb's current state.
4. The bulb responded instantly to these commands, demonstrating effective remote-controlled electrical automation.
  5. The 9V battery is crucial to the circuit's safe and reliable operation. The bulb and the relay require significantly higher voltage and current than the Arduino can safely provide from its 5V pin. So, the 9V battery acts as an independent power source to energize the relay coil and power the light bulb. This prevents overloading and potential damage to the Arduino's onboard components.
  6. The relay functions as an electrical isolator, allowing the low-power digital signals from the Arduino to safely and indirectly control the high-power bulb circuit.

## **7. Result:**

The simulated Remote AC Bulb Control system accurately detected and processed incoming IR signals. The Arduino efficiently converted remote button inputs into immediate actions, consistently turning the light bulb ON or OFF. This quick response verified the reliability of both the IR communication and the programmed control logic. Overall, the system clearly demonstrated remote-based automation using Arduino and highlighted the practical use of IR technology in smart home applications.

## **8. Conclusion:**

The practical setup clearly validated how remote-controlled electrical automation works in real applications. The system successfully incorporated an IR receiver to accurately decode incoming signals and employed a relay, driven by a separate 9V power supply, to securely operate a high-power load (the bulb) using low-power digital control. It demonstrated accurate and immediate responsiveness to the programmed commands from the remote, confirming reliable system performance.